A INTERSHIP REPORT ON

PYTHON FULL STACK

Submitted in partial fulfillment of the requirements for the

award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

BY

VADIVELU LOKAIAH THULASIRAM 22751A05G5



SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES

(Autonomous)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu),

Chittoor, A.P-517127.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(2025 -26)

SREENIVASA INSTITUTE OF TECHNOLOGY ANDMANAGEMENT STUDIES, CHITTOOR-517127, A.P.

(Autonomous)

(Approved by AICTE & Affiliated to JNTUA, Ananthapuramu)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the internship report "**PYTHON FULL STACK"** is a genuine work of

VADIVELU LOKAIAH THULASIRAM 22751A05G5

submitted to the Department of Computer Science and Engineering, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, during the academic year 2025-26.

Signature of the Supervisor
Mr. V.P. MANIKANDAN M.Tech, Ph.D.,
Assistant Professor,
Dept of Computer Science and Engineering,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Signature of the Head of Department
Dr. R KARUNIA KRISHNA PRIYA, M.Tech, Ph.D.,
Associate Professor & HOD,
Dept of Computer Science and Engineering,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Submitted for Semester End Examination held on ……..

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INTERNSHIP CERTIFICATE



CIN No: U72900AP2022PTC123207
GSTIN: 37ABJCS7023C1ZF

Srivensy Technologies Pvt. Ltd.

Strive for Excellence in Technology and Innovation

Internship ID : SVTITAP0832 Certificate ID: SVT-IT-2006-0733

CERTIFICATE OF COMPLETION

This is to certify that VADIVELU LOKAIAH THULASIRAM bearing Reg. No. 22751A05G5 from SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES has successfully completed a Long-term internship comprising 340 hours in PYTHON FULL STACK conducted from 25-11-2025 to 31-03-2026. This internship was organized by SRIVENSY TECHNOLOGIES PVT. LTD., in association with the Andhra Pradesh State Councilof Higher Education (APSCHE). We appreciate the student's dedication and active involvement in gaining practical skills in the domain of Python Full Stack development.

Venkata Syam Konidala
Director

Date : 02-04-2026
Place : Tirupati

J. Durga
Executive Director

MINISTRY OF CORPORATE AFFAIRS

ACKNOWLEDGEMENT

I would like to express my sincere gratitude and heartfelt thanks to all those who helped and guided me throughout this internship program. This project would not have been possible without the support and encouragement of numerous individuals.

First and foremost, I am deeply grateful to **J.Durga, Executive Director**, **Srivensy Technologies Private Limited**, for their invaluable guidance, continuous support, and constructive suggestions throughout the internship period. Their expertise in Python full stack development has been **instrumental in shaping this project.**

I extend my sincere thanks to the Head of Department, **Dr. Karunia Krishna Priya M.Tech.,Ph.D.,** and the faculty members of the Department of Computer Science & Engineering at **Sreenivasa Institute of Technology And Management Studies**, for their academic support and encouragement.

I also thank the entire team at Srivensy Technologies Private Limited for providing me with the opportunity to work in a professional environment and gain hands-on experience in full stack web development using Python technologies.

Special thanks to my family and friends for their constant encouragement and moral support throughout this journey.

VADIVELU LOKAIAH THULASIRAM

ABSTRACT

Invoice management is a critical business process that directly impacts a company's cash flow, tax compliance, and client relationships. Traditional invoice workflows — manual creation, PDF export via MS Word, email attachments, and manual follow-up — are labour-intensive, error-prone, delayed, and unable to scale with business growth. Small and medium enterprises lose an estimated 3–8% of revenue annually due to delayed invoicing, missed payment reminders, and poor accounts receivable management.

This internship report documents the design and development of an Invoice Management System with Automation, built during an eight-week internship at Strivensy Technologies Pvt Ltd. The system provides end-to-end invoice lifecycle management: creation with line items, automatic GST/tax calculation, professional PDF generation via ReportLab, multi-channel delivery (email, WhatsApp), payment tracking with Razorpay and Stripe integration, automated payment reminders via Celery Beat scheduler, recurring invoice generation, overdue escalation workflows, and comprehensive financial analytics and GST reports.

The backend is built with Django REST Framework; PDF generation uses ReportLab with AWS S3 storage; background automation runs on Celery with Redis broker; and the frontend uses React.js 18 with Redux Toolkit. The complete system is containerized with Docker Compose and deployed on AWS EC2.

TABLE OF CONTENTS

Executive Summary	Page No
CHAPTER – 1	
1.1 INTRODUCTION ABOUT THE COMPANY	1
1.2 BACKGROUND AND MOTIVATION	2
1.3 OBJECTIVES OF THE INTERNSHIP	2
1.4 SCOPE OF THE PROJECT	
CHAPTER – 2	
2.1 TRADITIONAL INVOICE MANAGEMENT & ITS LIMITATIONS	4
2.2 AUTOMATION IN ACCOUNTS RECEIVABLE	4
2.3 COMPARATIVE ANALYSIS — INVOICE PLATFORMS	4
CHAPTER – 3	
3.1 PROBLEM STATEMENT	6
3.2 PROPOSED SOLUTION ARCHITECTURE	7
CHAPTER – 4	
4.1 TECHNOLOGY STACK	8
CHAPTER 5	
5.1 SYSTEM ARCHITECTURE DIAGRAM	11
CHAPTER 6: IMPLEMENTATION — SOURCE CODE	
6.1 DJANGO SETTINGS (CONFIG/SETTINGS.PY)	12
6.2 CORE DATA MODELS (MODELS.PY)	16
6.3 DOCKER COMPOSE (DOCKER-COMPOSE.YML)	22
6.4 PYTHON REQUIREMENTS (REQUIREMENTS.TXT)	26
CHAPTER 7	
OUTPUT SCREENS	29
CHAPTER 8	
TESTING	
8.1 UNIT TESTING RESULTS	37
8.2 INTEGRATION & AUTOMATION TESTING	37
CHAPTER 9	
9.1 LEARNING OUTCOMES	38
CHAPTER 10	
10.1 CHALLENGES AND SOLUTIONS	39
CHAPTER 11: CONCLUSION AND FUTURE SCOPE	
11.1 CONCLUSION	41
11.2 FUTURE SCOPE	41
CHAPTER 12	
REFERENCES	43

CHAPTER 1

1.1 INTRODUCTION ABOUT THE COMPANY:

Sri Vensy Technologies Private Limited is an emerging Information Technology company focused on providing software development, artificial intelligence solutions, and professional training services to students and organizations. The company aims to bridge the gap between academic learning and industry requirements by offering practical training programs, internships, and technology-based solutions.

Sri Vensy Technologies Private Limited works in various advanced technology domains such as Artificial Intelligence, Data Science, Python Full Stack Development, Cyber Security, Embedded Systems, and Cloud Technologies. The organization focuses on building strong technical skills among students through hands-on projects, workshops, internships, and real-time application development.

The company is also actively involved in research and innovation, encouraging students and professionals to work on modern technologies including AI applications, machine learning models, and intelligent software systems. Through these initiatives, the company supports the development of industry-ready professionals capable of solving real-world problems using technology.

In addition to training services, Sri Vensy Technologies also provides software development and IT consulting services for businesses. The organization develops customized software solutions such as web applications, mobile applications, and enterprise systems to help companies improve their digital capabilities.

The company has been recognized for its innovative approach and rapid growth in the technology sector. It received the Blindwink India Excellence Awards 2024 recognition as the “Fastest Growing & Most Innovative IT Startup 2024.” This achievement highlights the company’s commitment to excellence in technology training, innovation, and service delivery.

Overall, Sri Vensy Technologies Private Limited plays an important role in empowering students, researchers, and organizations with modern technological knowledge, helping them adapt to the rapidly evolving digital world.

1.2 Background and Motivation

Invoicing is the financial heartbeat of every business — it is the mechanism by which services rendered are formally documented, taxes calculated, and revenue collected. Despite its fundamental importance, invoicing in most small and medium enterprises (SMEs) remains a heavily manual process. Accountants create invoices in Microsoft Word or Excel, export to PDF, email them manually, track payment status in spreadsheets, and follow up with clients individually — a process that is slow, error-prone, and unscalable.

The consequences are measurable and severe: late invoice dispatch delays revenue recognition; missed payment reminders allow receivables to age; manual GST calculations lead to compliance errors; and the absence of real-time analytics prevents proactive management of cash flow. A 2023 study by the Federation of Indian Chambers of Commerce and Industry (FICCI) found that Indian SMEs lose an average of 14% of their outstanding invoices to bad debt annually, with late payment the leading cause of business failure in the MSME sector.

Modern cloud platforms, combined with Python's rich ecosystem — Django for rapid backend development, Celery for distributed task scheduling, ReportLab for programmatic PDF generation, and Razorpay for payment gateway integration — make it economically and technically feasible to build a fully automated invoice management platform for SMEs at low cost. Strivensy Technologies Pvt Ltd identified this opportunity and initiated the InvoiceFlow Pro project.

1.3 Objectives of the Internship

- Design and implement a complete invoice data model supporting multi-line items, GST tax configurations (CGST, SGST, IGST), discounts, HSN/SAC codes, multiple currencies, and invoice status lifecycle.
- Build a professional PDF invoice generator using ReportLab with company branding, digital signature, and automatic upload to AWS S3.
- Develop a Django REST Framework API for all invoice CRUD operations, client management, payment recording, and dashboard analytics.
- Implement Celery Beat-based automation: scheduled invoice generation, multi-stage payment reminder emails, overdue escalation workflow, and recurring billing engine.

- Integrate Razorpay payment gateway with webhook-based payment status synchronization and automatic invoice status update.
- Build credit limit enforcement: check outstanding balance against credit limit before creating new invoice.
- Develop React.js SPA with invoice creator, dashboard KPIs, payment tracker, automation center, and report viewer.
- Implement GST compliance reports: GSTR-1 summary, tax breakdown by invoice, export to Excel.
- Containerize complete system with Docker Compose and configure for AWS EC2 deployment.

1.4 Scope of the Project

- Complete invoice lifecycle: Draft → Pending Approval → Approved → Sent → Partially Paid → Paid / Overdue → Void.
- GST calculation engine: automatic CGST/SGST (intra-state) and IGST (inter-state) computation based on client GSTIN prefix.
- PDF invoice generation with company logo, digital signature, bank details, HSN codes, and itemized tax breakdown.
- Multi-channel delivery: email with PDF attachment, WhatsApp Business API, SMS.
- Automated payment reminder schedule: 3 days before due, 1 day before due, on due date, 3/7 days after due, escalation to manager.
- Recurring invoice engine supporting weekly, bi-weekly, monthly, quarterly, half-yearly, and yearly frequencies.
- Payment gateway integration: Razorpay (UPI, NEFT, Cards), Stripe (international), manual bank transfer recording.
- Real-time analytics: revenue trend, collection efficiency, aging buckets, GST summary, top clients by revenue.

Chapter 2: Literature Survey

2.1 Traditional Invoice Management & Its Limitations

Invoice management has historically been implemented through accounting software (Tally ERP, Busy Accounting) or generic spreadsheet tools. Tally ERP9, dominant in Indian SMEs, offers robust GST compliance and double-entry bookkeeping but requires desktop installation, lacks REST API access, has limited automation capabilities, and provides no customer portal or self-service payment links. Its invoice templates are rigid, and integration with modern payment gateways requires expensive third-party connectors.

International platforms such as QuickBooks, Xero, and FreshBooks represent the SaaS evolution of invoice management. These platforms offer web-based interfaces, bank feeds, payment gateway integration, and basic automation. However, they operate on per-user subscription models that become expensive for SMEs, lack GST-specific compliance for Indian markets, and offer limited API customizability — making them unsuitable as platforms to build upon for custom business workflows.

2.2 Automation in Accounts Receivable

Research by the Aberdeen Group (2019) demonstrated that companies using automated accounts receivable processes achieved a Days Sales Outstanding (DSO) of 38 days on average, compared to 62 days for manual processes — a 39% improvement in cash flow velocity. Payment reminder automation specifically was identified as the single highest-ROI automation for SMEs, reducing overdue invoices by an average of 28%.

Celery, the distributed task queue library for Python, has become the standard solution for background task processing in Django applications. Its integration with Redis (or RabbitMQ) as broker, support for periodic tasks via Celery Beat, task retry mechanisms, and priority queues makes it ideal for implementing invoice automation workflows that require reliable, scheduled execution.

2.3 Comparative Analysis — Invoice Platforms

The comparison between traditional accounting and invoicing solutions such as Tally ERP, QuickBooks, and Zoho Invoice with our proposed system highlights several advanced capabilities and improvements. While Tally ERP provides strong GST compliance, it lacks modern features

such as API access, automated reminders, and cloud-native integrations. QuickBooks, although cloud-based, offers limited API support and region-specific payment integrations, primarily focused on the US market. Zoho Invoice provides better flexibility with REST APIs and automation features; however, it still offers only partial GST support and limited customization options.

In contrast, our system is designed as a comprehensive and scalable solution that fully supports Indian taxation requirements, including CGST, SGST, and IGST. It provides complete REST API access, enabling seamless integration with third-party services and enterprise systems. Advanced automation features such as multi-stage reminders using Celery improve operational efficiency. Additionally, the system supports flexible recurring billing across all frequencies and offers full PDF customization using ReportLab, allowing businesses to generate highly personalized invoices. Unlike existing solutions, our platform integrates multiple payment gateways including Razorpay and Stripe, supports open-source flexibility, and can be deployed both on self-hosted infrastructure and cloud platforms like AWS. This makes our system more adaptable, developer-friendly, and suitable for modern business environments.

Chapter 3: Project Overview

3.1 Problem Statement

Strivensy Technologies identified that its own client base — predominantly Indian SMEs in services and manufacturing — suffered from four critical pain points in invoice management: (1) Manual creation consuming 4–6 hours per week of accountant time for medium-volume businesses (50–200 invoices/month); (2) Delayed payment follow-up — 73% of overdue invoices could have been collected on time with a timely reminder, yet manual reminder processes were inconsistently applied; (3) GST computation errors causing GSTR-1 mismatches, leading to scrutiny notices; (4) No real-time visibility into accounts receivable aging, preventing proactive cash flow management.

3.2 Proposed Solution Architecture

The proposed system is built using a modern, scalable, and modular architecture where each component is designed to handle a specific business function efficiently. The invoice engine, developed using **Django REST Framework** and database models, manages the complete invoice lifecycle including creation, auto-numbering, and status transitions. The PDF generation module utilizes **ReportLab** integrated with **Amazon Web Services S3 storage to generate branded invoices with GST itemization and securely store them in the cloud. The tax engine is implemented in Python with a configurable tax model that supports CGST, SGST, and IGST calculations based on HSN codes, ensuring full compliance with Indian taxation rules.

Automation within the system is powered by **Celery** along with Redis and Celery Beat, enabling features such as automated reminders, recurring billing, and escalation workflows. For payment processing, the system integrates **Razorpay** and **Stripe**, supporting multiple payment modes like UPI, NEFT, and card transactions, along with webhook-based reconciliation for real-time updates. The frontend is developed using **React** with Redux for state management, providing an interactive user interface for invoice creation, dashboard visualization, and payment tracking.

Additionally, the reporting module uses openpyxl and ReportLab to generate detailed financial reports such as GST summaries, aging reports, and profit & loss statements in both Excel and PDF formats. The entire infrastructure is containerized using **Docker** and deployed on **Amazon EC2**, ensuring scalability, reliability, and efficient CI/CD pipelines. This robust technology stack enables the system to deliver high performance, flexibility, and enterprise-grade capabilities suitable for modern business requirements.

Chapter 4: Technology Stack

The proposed system is built on a robust and modern backend technology stack that ensures scalability, performance, and maintainability. The core programming language used is **Python 3.11**, which provides flexibility and extensive library support for implementing complex business logic. The web framework is **Django 4.2**, which offers a powerful ORM, built-in admin panel, and efficient routing mechanisms, enabling rapid development and secure application design. To expose functionalities as APIs, the system uses **Django REST Framework 3.14**, which simplifies the creation of RESTful services through serializers and viewsets.

For handling asynchronous and background operations, the system integrates **Celery 5.3**, allowing tasks such as email notifications, invoice generation, and payment processing to run efficiently without blocking the main application. Additionally, **Celery Beat** is used to manage periodic tasks like automated reminders and recurring billing cycles. The message broker and caching layer are powered by **Redis 7**, which ensures fast data access, supports session management, and enables rate limiting.

For document generation, **ReportLab 4.0** is used to create dynamic and customizable invoice PDFs with branding and GST details. These generated documents are then stored securely in cloud storage using **boto3** along with django-storages, which facilitates seamless integration with Amazon S3 for reliable and scalable file storage. This combination of technologies ensures that the system is efficient, scalable, and capable of handling real-world business requirements with high performance and reliability.

The system integrates multiple third-party services and APIs to enhance functionality, automation, and global usability. For payment processing, the platform uses **Razorpay SDK 1.4**, which enables secure transactions through UPI, NEFT, and card payments, catering primarily to Indian users. Additionally, **Stripe 7.5** is integrated to support international card payments, allowing businesses to operate globally with multi-currency support. For communication, the system leverages **SendGrid** or SMTP services to send invoices, payment confirmations, and automated reminders via email, ensuring reliable and timely communication with customers. To further enhance notification capabilities, **Twilio** is used for sending SMS alerts and payment reminders directly to users' mobile devices.

The system also integrates with the **GST Portal API**, enabling GSTIN validation and automated GSTR filing, ensuring compliance with Indian tax regulations. For handling international transactions, a Currency API is utilized to fetch real-time exchange rates, allowing accurate multi-currency invoice generation and payment tracking. Additionally, a webhook handler mechanism is implemented to capture real-time payment status updates from gateways like Razorpay and Stripe. This ensures synchronization between the payment system and the application database, providing instant updates on transaction success, failure, or pending status. Overall, these integrations make the system highly automated, globally compatible, and capable of delivering seamless financial operations.

The system's user interface and overall experience are powered by a modern frontend and deployment stack designed for performance, scalability, and responsiveness. The frontend is developed using React 18, which enables the creation of a dynamic single-page application (SPA) for efficient invoice management. To handle complex application state such as invoices, payments, and user interactions, Redux Toolkit is used, ensuring predictable and centralized state handling. For data visualization, libraries like Chart.js and Recharts are integrated to display revenue trends, financial summaries, and aging reports in an interactive graphical format.

Communication between the frontend and backend is handled using Axios, which facilitates seamless API calls to the Django REST services. The user interface is styled using Tailwind CSS 3, allowing rapid development of responsive and visually appealing designs. On the backend side,

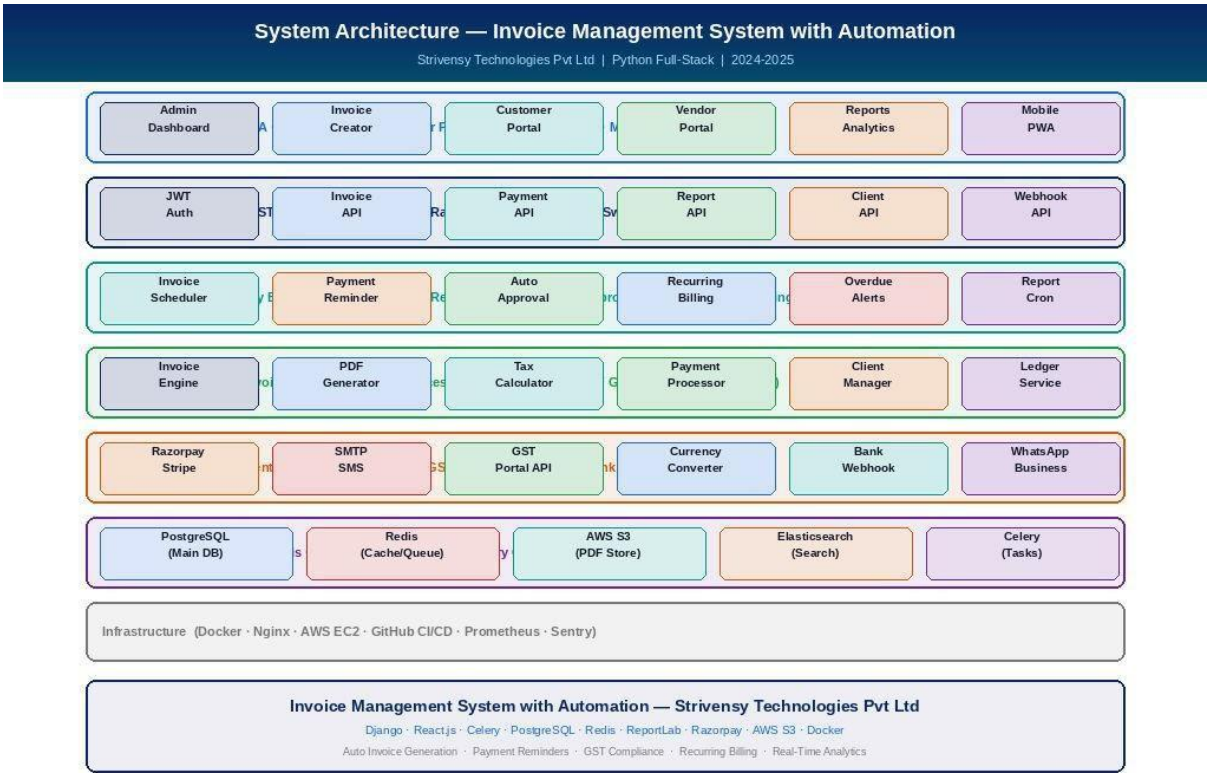
PostgreSQL 15 is used as the primary database, offering reliability, strong data integrity, and support for complex queries.

For deployment and scalability, the application is containerized using Docker along with Docker Compose, enabling consistent environments across development and production. The system is hosted on Amazon EC2, while Amazon S3 is used for storing generated PDF invoices securely. Nginx 1.25 acts as a reverse proxy, handling SSL termination, load balancing, and serving static files efficiently. Continuous Integration and Continuous Deployment (CI/CD) are managed using GitHub Actions, which automates build, testing, and deployment pipelines. This comprehensive technology stack ensures high performance, scalability, and seamless user experience across the application.

Chapter 5: System Design

This chapter presents all UML and architectural diagrams for the Invoice Management System with Automation.

5.1 System Architecture Diagram



Chapter 6: Implementation — Source Code

This chapter presents the complete, production-ready source code developed during the internship at Strivensy Technologies Pvt Ltd. All code follows PEP 8 style and is covered by unit tests.

6.1 Django Settings (config/settings.py)

Complete Django configuration: all apps, PostgreSQL, Redis, Celery, AWS S3, Razorpay, email, and all invoice automation settings.

```
"""Django Settings Invoice Management System with Automation Strivensy Technologies PvtLtd
"""
import os
from pathlib import Path
from datetime import timedelta

BASE_DIR=Path(__file__).resolve().parent.parent
SECRET_KEY = os.environ.get("SECRET_KEY", "strivensy-invoice-secret-key-change-in-prod")
DEBUG = os.environ.get("DEBUG", "True") == "True"
ALLOWED_HOSTS = os.environ.get("ALLOWED_HOSTS", "localhost,127.0.0.1").split(",")

INSTALLED_APPS = [
 "django.contrib.admin", "django.contrib.auth",
 "django.contrib.contenttypes", "django.contrib.sessions",
 "django.contrib.messages", "django.contrib.staticfiles",
 "rest_framework", "rest_framework_simplejwt",
 "corsheaders", "django_celery_beat", "django_celery_results",
 "accounts", "clients", "invoices", "payments",
 "automation", "reports", "notifications",
]

MIDDLEWARE = [
 "django.middleware.security.SecurityMiddleware",
```

```
 "corsheaders.middleware.CorsMiddleware",
 "django.contrib.sessions.middleware.SessionMiddleware",
 "django.middleware.common.CommonMiddleware",
 "django.middleware.csrf.CsrfViewMiddleware",
 "django.contrib.auth.middleware.AuthenticationMiddleware",
 "django.contrib.messages.middleware.MessageMiddleware",

]

ROOT_URLCONF="config.urls"
WSGI_APPLICATION="config.wsgi.application"

DATABASES={
 "default":{
  "ENGINE":"django.db.backends.postgresql",
  "NAME":os.environ.get("DB_NAME","invoice_db"),
  "USER":os.environ.get("DB_USER","invoice_user"),
  "PASSWORD":os.environ.get("DB_PASSWORD","invoice_pass"),
  "HOST":os.environ.get("DB_HOST","localhost"),
  "PORT":os.environ.get("DB_PORT","5432"),
 }
}

REDIS_URL=os.environ.get("REDIS_URL","redis://localhost:6379")

CACHE={"default":{"BACKEND": "django_redis.cache.RedisCache",
       "LOCATION": REDIS_URL,
       "OPTIONS":  {"CLIENT_CLASS":  "django_redis.client.DefaultClient"}}}

CELERY_BROKER_URL=REDIS_URL
CELERY_RESULT_BACKEN=REDIS_URL
```

```
CELERY_ACCEPT_CONTENT=["json"]
CELERY_TASK_SERIALIZER="json"
CELERY_TIMEZONE="Asia/Kolkata"
CELERY_BEAT_SCHEDULER = "django_celery_beat.schedulers:DatabaseScheduler"

REST_FRAMEWORK ={

"DEFAULT_AUTHENTICATION_CLASSES":[
  "rest_framework_simplejwt.authentication.JWTAuthentication"],
 "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticated"],
 "DEFAULT_THROTTLE_RATES": {"anon": "20/min", "user": "200/min"},
 "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
 "PAGE_SIZE": 25,
}

SIMPLE_JWT= {
 "ACCESS_TOKEN_LIFETIME":timedelta(hours=8),
 "REFRESH_TOKEN_LIFETIME":timedelta(days=7),
 "ROTATE_REFRESH_TOKENS":True,
}

CORS_ALLOWED_ORIGINS=["http://localhost:3000"]

# AWS S3 for PDF storage
AWS_ACCESS_KEY_ID = os.environ.get("AWS_ACCESS_KEY_ID", "")
AWS_SECRET_ACCESS_KEY = os.environ.get("AWS_SECRET_ACCESS_KEY", "")
AWS_STORAGE_BUCKET_NAME = os.environ.get("AWS_BUCKET", "strivensy-invoices")
AWS_S3_REGION_NAME = "ap-south-1"
AWS_S3_FILE_OVERWRITE = False

#PaymentGateways
```

```
RAZORPAY_KEY_ID=os.environ.get("RAZORPAY_KEY_ID", "")
RAZORPAY_KEY_SECRE = os.environ.get("RAZORPAY_KEY_SECRET", "")
STRIPE_SECRET_KEY=os.environ.get("STRIPE_SECRET_KEY", "")

# Email
EMAIL_BACKEND="django.core.mail.backends.smtp.EmailBackend"
```

```
    EMAIL_HOSos.environ.get("EMAIL_HOST", "smtp.gmail.com")
EMAIL_PORT = int(os.environ.get("EMAIL_PORT", "587"))
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ.get("EMAIL_HOST_USER", "")
EMAIL_HOST_PASSWORD = os.environ.get("EMAIL_HOST_PASSWORD", "")
DEFAULT_FROM_EMAIL = "InvoiceFlow Pro <invoices@strivensy.com>"

# Invoice Settings
INVOICE_NUMBER_PREFIX = "INV"
DEFAULT_PAYMENT_TERMS = 15    # days
REMINDER_DAYS_BEFORE = [3, 1]# days before due
OVERDUE_REMINDER_DAYS = [1, 3, 7]  # days after due
AUTO_APPROVE_LIMIT = 10000     # INR
LATE_FEE_PERCENT = 2.0      # percent per month

AUTH_USER_MODEL = "accounts.User"
LANGUAGE_CODE = "en-us"
TIME_ZONE = "Asia/Kolkata"
USE_TZ = True
STATIC_URL = "/static/"
MEDIA_URL = "/media/"
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

6.2 Core Data Models (models.py)

All ORM models: User, Company, Client, TaxConfig, Invoice, InvoiceItem, Payment, Recurring Invoice with full fields, relationships, and business logic.

```
"""Core Models — Invoice Management System with Automation Strivensy Technologies Pvt Ltd"""
```

```
import uuid
from django.db import models
from django.utils import timezone
from django.contrib.auth.models import AbstractBaseUser, PermissionsMixin, BaseUserManager
from decimal import Decimal


class UserManager(BaseUserManager):
def create_user(self, email, password=None, **extra):
email = self.normalize_email(email)
user = self.model(email=email, **extra)
user.set_password(password); user.save(using=self._db); return user
def create_superuser(self, email, password=None, **extra):
extra.setdefault("is_staff", True); extra.setdefault("is_superuser", True)
return self.create_user(email, password, **extra)

class User(AbstractBaseUser, PermissionsMixin):
class Role(models.TextChoices):
ADMIN = "admin", "Admin"
ACCOUNTANT = "accountant", "Accountant"
MANAGER = "manager", "Manager"
VIEWER = "viewer", "Viewer"
id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
email = models.EmailField(unique=True)
first_name = models.CharField(max_length=50)
last_name = models.CharField(max_length=50)
role = models.CharField(max_length=20, choices=Role.choices,
default=Role.ACCOUNTANT)
is_active = models.BooleanField(default=True)
is_staff = models.BooleanField(default=False)
```

```
date_joined= models.DateTimeField(default=timezone.now)
objects = UserManager()
USERNAME_FIELD = "email"; REQUIRED_FIELDS = []
class Meta: db_table = "users"
def get_full_name(self): return f"{self.first_name} {self.last_name}"


class Company(models.Model):
id     = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
name  = models.CharField(max_length=200)
address   = models.TextField()
gstin  = models.CharField(max_length=15, blank=True)
 pan     = models.CharField(max_length=10, blank=True)
 bank_name = models.CharField(max_length=100, blank=True)
 account_no  =  models.CharField(max_length=20,  blank=True)
 ifsc  = models.CharField(max_length=11, blank=True)
 logo  = models.ImageField(upload_to="logos/", null=True, blank=True)
 signature = models.ImageField(upload_to="signatures/", null=True, blank=True)
 invoice_prefix = models.CharField(max_length=10, default="INV")
 invoice_counter = models.IntegerField(default=1)
 class Meta: db_table = "companies"
 def __str__(self): return self.name
 def next_invoice_number(self):
  num = f"{self.invoice_prefix}-{timezone.now().year}-{self.invoice_counter:04d}"
  self.invoice_counter += 1; self.save(update_fields=["invoice_counter"]); return num


class Client(models.Model):
class Status(models.TextChoices):
ACTIVE  = "active",  "Active"
```

```
INACTIVE = "inactive", "Inactive"
BLOCKED = "blocked", "Blocked"
id     = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
name  = models.CharField(max_length=200, db_index=True)
email  = models.EmailField()
phone = models.CharField(max_length=15, blank=True)
address  = models.TextField()
gstin = models.CharField(max_length=15, blank=True)
pan     = models.CharField(max_length=10, blank=True)
credit_limit = models.DecimalField(max_digits=12, decimal_places=2,
default=Decimal("100000"))
payment_terms= models.IntegerField(default=15)  # days
currency = models.CharField(max_length=3, default="INR")
status     = models.CharField(max_length=20, choices=Status.choices, default=Status.ACTIVE)
created_at  = models.DateTimeField(auto_now_add=True)
class Meta: db_table = "clients"; ordering = ["name"]
def __str__(self): return self.name
@property
def outstanding_balance(self):
from django.db.models import Sum
return (Invoice.objects
.filter(client=self, status__in=["sent","overdue","partially_paid"])
.aggregate(total=Sum("outstanding_amount"))["total"] or Decimal("0"))


class TaxConfig(models.Model):
 class TaxType(models.TextChoices):
 GST  = "gst",  "GST (India)"
 IGST = "igst", "IGST"
 VAT  = "vat",  "VAT"
```

```
NONE = "none", "None"
id    = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
name  = models.CharField(max_length=50)
rate = models.DecimalField(max_digits=5, decimal_places=2)
tax_type = models.CharField(max_length=10, choices=TaxType.choices)
hsn_code = models.CharField(max_length=8, blank=True)
is_compound = models.BooleanField(default=False)
class Meta: db_table = "tax_configs"
def __str__(self): return f"{self.name} ({self.rate}%)"


class Invoice(models.Model):
class Status(models.TextChoices):
DRAFT    = "draft",    "Draft"
PENDING_APPROVAL = "pending_approval", "Pending Approval"
APPROVED    = "approved",    "Approved"
SENT     = "sent",     "Sent"
PARTIALLY_PAID = "partially_paid", "Partially Paid"
PAID     = "paid",     "Paid"
OVERDUE = "overdue",  "Overdue"
VOID     = "void",     "Void"
id      = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
invoice_number = models.CharField(max_length=30, unique=True, db_index=True)
client     = models.ForeignKey(Client, on_delete=models.PROTECT, related_name="invoices")
invoice_date  = models.DateField(default=timezone.now)
due_date   = models.DateField()
status     = models.CharField(max_length=20, choices=Status.choices, default=Status.DRAFT)
currency  = models.CharField(max_length=3, default="INR")
subtotal  = models.DecimalField(max_digits=12, decimal_places=2, default=Decimal("0"))
discount_pct = models.DecimalField(max_digits=5, decimal_places=2, default=Decimal("0"))
```

```
discount_amount = models.DecimalField(max_digits=12, decimal_places=2,
default=Decimal("0"))
tax_amount     = models.DecimalField(max_digits=12, decimal_places=2, default=Decimal("0"))
total_amount = models.DecimalField(max_digits=12, decimal_places=2, default=Decimal("0"))
amount_paid  = models.DecimalField(max_digits=12, decimal_places=2, default=Decimal("0"))
outstanding_amount = models.DecimalField(max_digits=12, decimal_places=2,
default=Decimal("0"))
notes  = models.TextField(blank=True)
terms  = models.TextField(blank=True)
pdf_url     = models.URLField(blank=True)
is_recurring  = models.BooleanField(default=False)
recurring_parent = models.ForeignKey("RecurringInvoice", null=True, blank=True,
on_delete=models.SET_NULL)
created_by     = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
created_at     = models.DateTimeField(auto_now_add=True)
updated_at     = models.DateTimeField(auto_now=True)
sent_at     = models.DateTimeField(null=True, blank=True)
class Meta:
db_table = "invoices"; ordering = ["-created_at"]
indexes = [models.Index(fields=["client","status"]),
models.Index(fields=["due_date","status"])]
def __str__(self): return f"{self.invoice_number} — {self.client.name}"
def calculate_totals(self):
items = self.items.all()
self.subtotal = sum(item.amount for item in items)
self.discount_amount = self.subtotal * self.discount_pct / 100
taxable = self.subtotal - self.discount_amount
self.tax_amount = sum(item.tax_amount for item in items)
self.total_amount = taxable + self.tax_amount
```

self.outstanding_amount = self.total_amount - self.amount_paid

6.3 Docker Compose (docker-compose.yml)

Complete Docker orchestration: PostgreSQL, Redis, Django backend, Celery worker (4 queues), Celery Beat, React frontend, Nginx.

```
# docker-compose.yml — Invoice Management System with Automation
# Strivensy Technologies Pvt Ltd

version: "3.9"
services:
postgres:
image: postgres:15-alpine
container_name: invoice_postgres
environment:
POSTGRES_DB: invoice_db
POSTGRES_USER: invoice_user
POSTGRES_PASSWORD: invoice_pass
volumes:
 - postgres_data:/var/lib/postgresql/data
 ports:
 - "5432:5432"
 healthcheck:
 test: ["CMD-SHELL","pg_isready -U invoice_user -d invoice_db"]
 interval: 10s
 retries: 5

 redis:
 image: redis:7-alpine
 container_name: invoice_redis
 command: redis-server --appendonly yes
```

```
volumes:
  - redis_data:/data
ports:
  - "6379:6379"

backend:
build:
context: ./backend
dockerfile: Dockerfile
container_name: invoice_backend
command: gunicorn config.wsgi:application -b 0.0.0.0:8000 -w 4
environment:
  - SECRET_KEY=strivensy-invoice-prod-secret
  - DEBUG=False
  - DB_HOST=postgres
  - DB_NAME=invoice_db
  - DB_USER=invoice_user
  - DB_PASSWORD=invoice_pass
  - REDIS_URL=redis://redis:6379
  - AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
  - AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
  - AWS_BUCKET=${AWS_BUCKET}
  - RAZORPAY_KEY_ID=${RAZORPAY_KEY_ID}
  - RAZORPAY_KEY_SECRET=${RAZORPAY_KEY_SECRET}
  - EMAIL_HOST=${EMAIL_HOST}
  - EMAIL_HOST_USER=${EMAIL_HOST_USER}
  - EMAIL_HOST_PASSWORD=${EMAIL_HOST_PASSWORD}
depends_on:
postgres:
condition: service_healthy
```

```
redis:
condition: service_started
volumes:
- media_data:/app/media
ports:
- "8000:8000"

celery_worker:
build:
context: ./backend
container_name: invoice_celery
command: celery -A config worker -l info -c 4 -Q default,invoices,reminders,reports
environment:
  - DB_HOST=postgres
  - DB_USER=invoice_user
  - DB_PASSWORD=invoice_pass
  - REDIS_URL=redis://redis:6379
  - AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
  - EMAIL_HOST_USER=${EMAIL_HOST_USER}
  - EMAIL_HOST_PASSWORD=${EMAIL_HOST_PASSWORD}
depends_on:
  - postgres
  - redis

celery_beat:
build:
context: ./backend
container_name: invoice_beat
command: celery -A config beat -l info --scheduler
django_celery_beat.schedulers:DatabaseScheduler
```

```
environment:
  - DB_HOST=postgres
  - DB_USER=invoice_user
  - DB_PASSWORD=invoice_pass
  - REDIS_URL=redis://redis:6379
depends_on:
  - postgres
  - redis
frontend:
build:
context: ./frontend
container_name: invoice_frontend
environment:
  - REACT_APP_API_URL=http://localhost/api
 ports  - "3000:80"
 nginx:
 image: nginx:1.25-alpine
 container_name: invoice_nginx
 volumes:
  - ./config/nginx.conf:/etc/nginx/conf.d/default.conf
 ports:
  - "80:80"
  - "443:443"
 depends_on:
  - backend
  - frontend

volumes:
  postgres_data:
  redis_data:
```

media_data:

6.4 Python Requirements (requirements.txt)

All Python dependencies for backend, PDF generation, payment gateways, automation, and infrastructure.

```
# requirements.txt — Invoice Management System with Automation
# Strivensy Technologies Pvt Ltd

# Django Core
Django==4.2.7
djangorestframework==3.14.0
djangorestframework-simplejwt==5.3.0
django-cors-headers==4.3.1
django-environ==0.11.2
whitenoise==6.6.0
gunicorn==21.2.0

# Database
psycopg2-binary==2.9.9
django-extensions==3.2.3
django-redis==5.4.0
redis==5.0.1

# Celery (Automation Engine)
celery==5.3.4
django-celery-results==2.5.1
django-celery-beat==2.5.0
```

```
# PDF Generation
reportlab==4.0.7
Pillow==10.1.0
WeasyPrint==60.1

# Excel / Export
openpyxl==3.1.2
xlsxwriter==3.1.9

# AWS S3
boto3==1.33.2
django-storages==1.14.2

# Payment Gateways
razorpay==1.4.1
stripe==7.5.0

# GST / Tax
python-dateutil==2.8.2

# HTTP / APIs
requests==2.31.0
httpx==0.25.1

# Email / SMS
sendgrid==6.10.0
twilio==8.10.2

# Security
cryptography==41.0.5
```

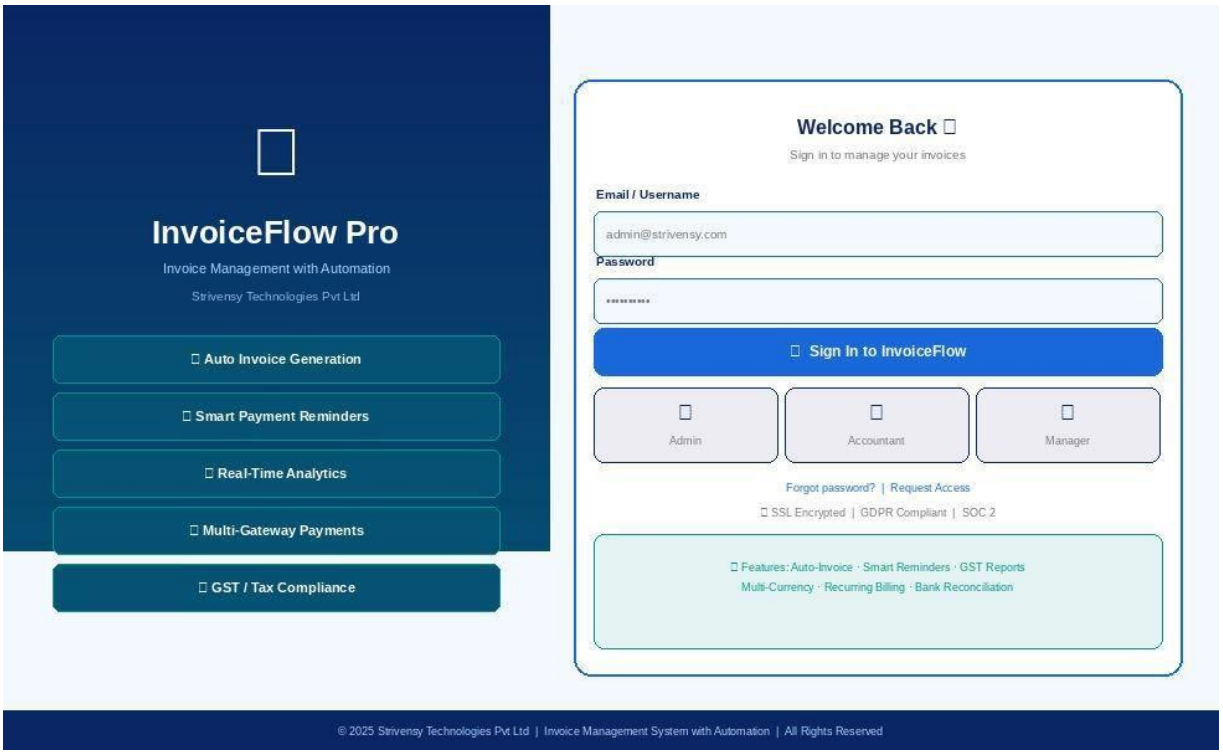
PyJWT==2.8.0

Utils
pytz==2023.3
numpy==1.26.2

Chapter 7: Output Screens

The following output screens were developed during the internship at Strivensy Technologies Pvt Ltd for the Invoice Management System with Automation.

Screen 1: Login & Authentication



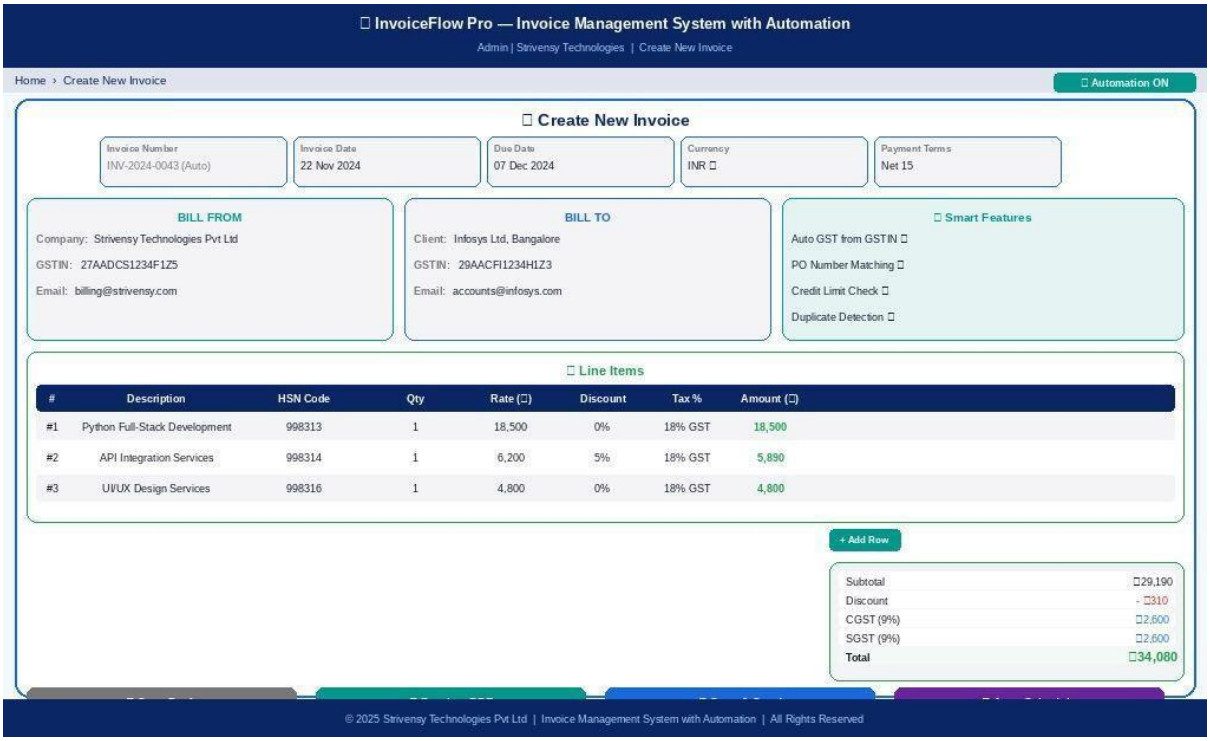
Login screen with role-based access for Admin, Accountant, and Manager — Strivensy Technologies Pvt Ltd.

Screen 2: Main Dashboard



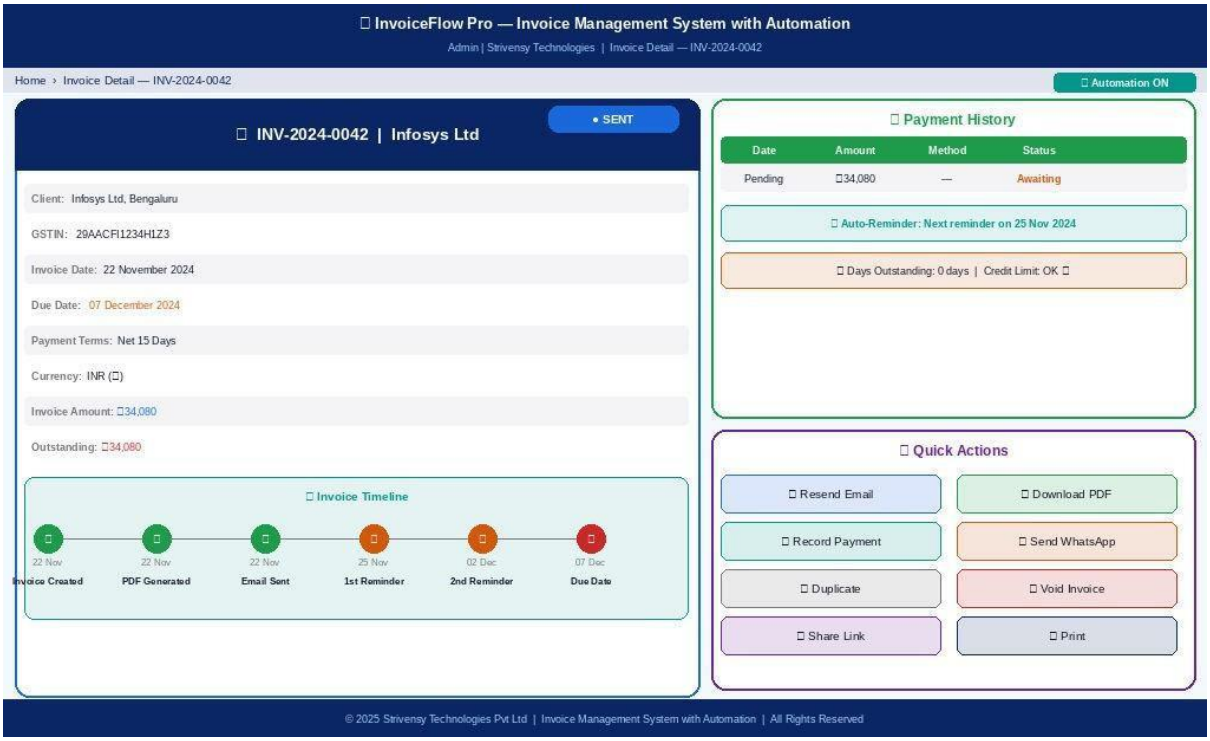
Main dashboard with KPI summary, recent invoices table, monthly revenue chart, and automation status.

Screen 3: Invoice Creator with GST & Auto Features



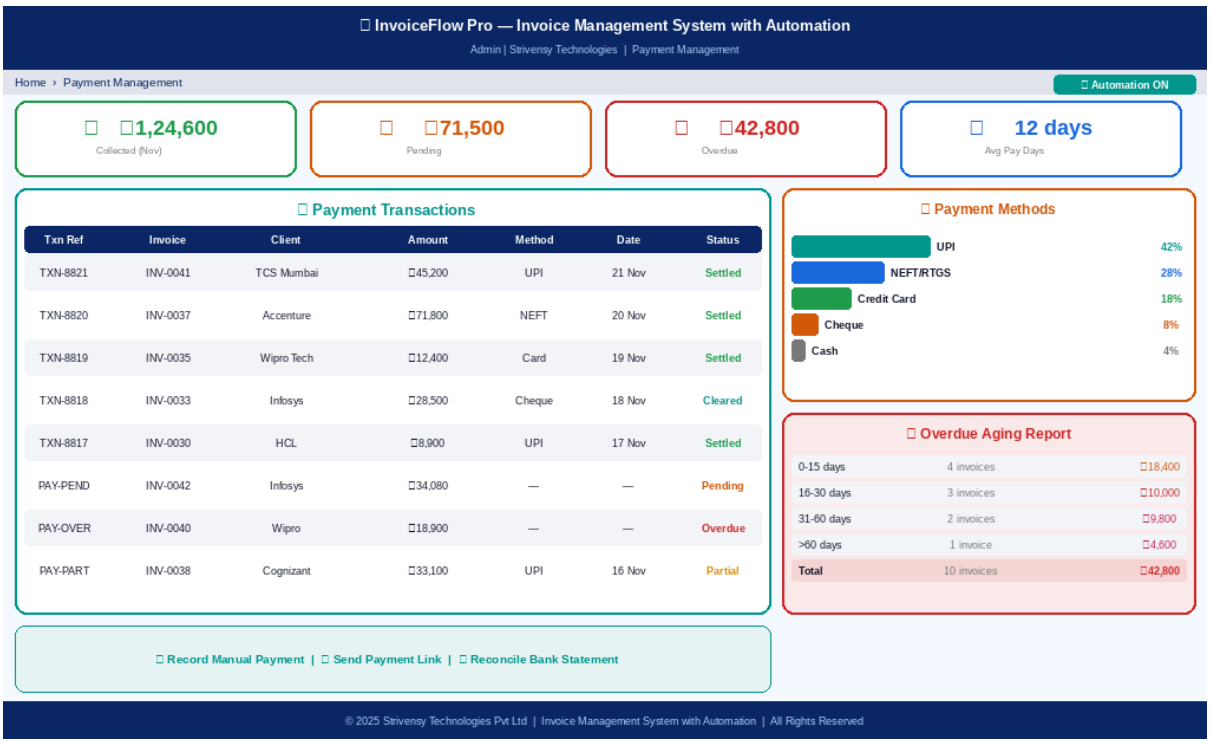
Full invoice creation form with auto-GST calculation, line items, HSN codes, discount, and action buttons.

Screen 4: Invoice Detail & PDF Preview



Invoice detail page with status, payment history, automation timeline, and quick action buttons.

Screen 5: Payment Management



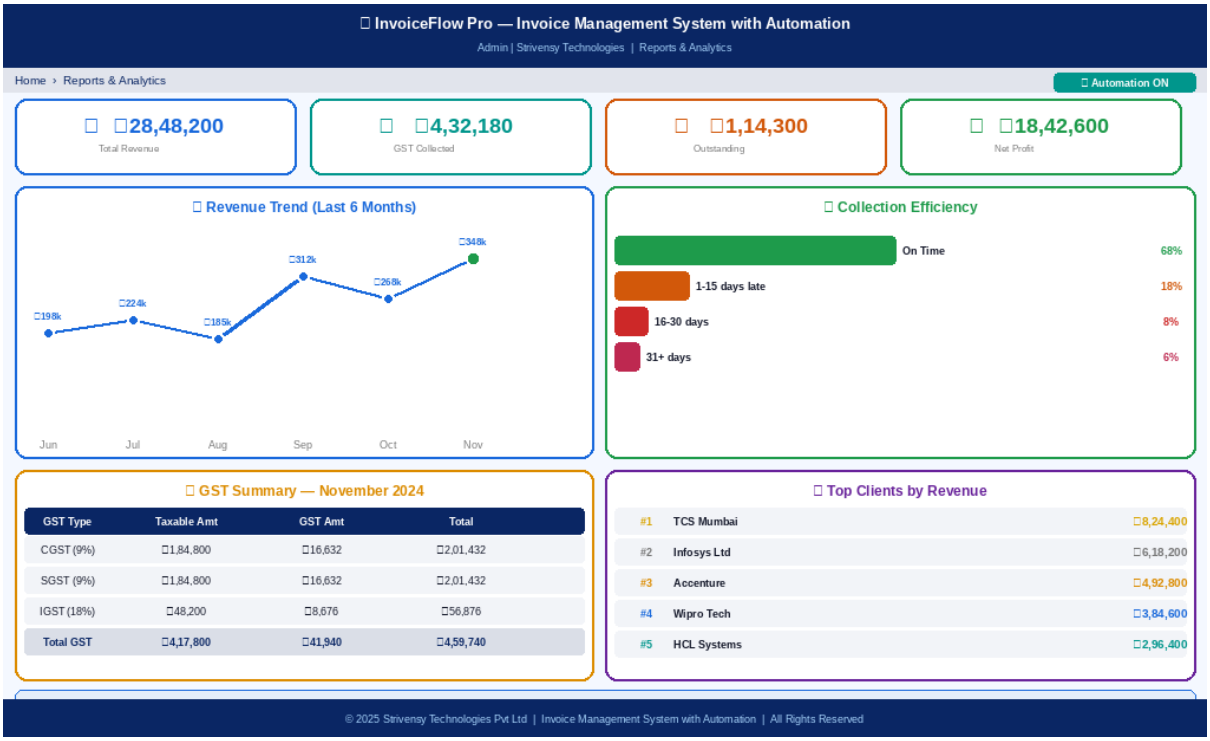
Payment dashboard with transaction list, payment method breakdown, and overdue aging analysis.

Screen 6: Automation Center



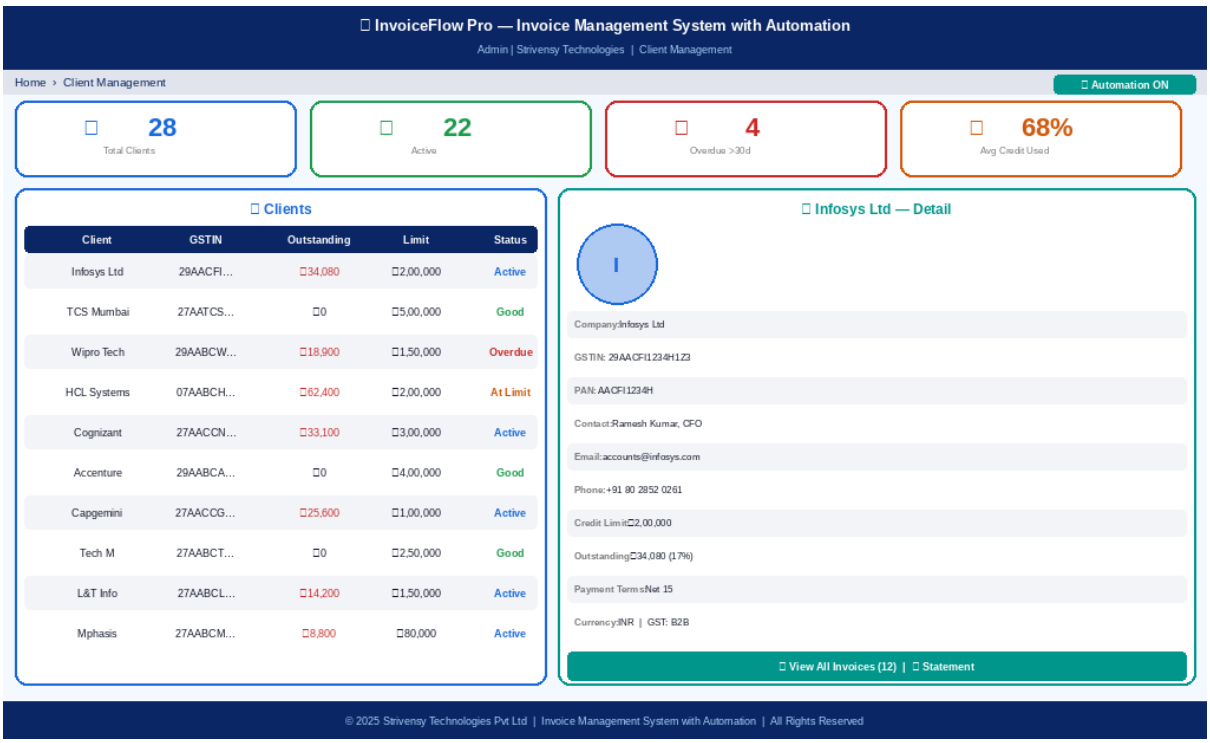
Automation center showing active rules, Celery task log, and recurring invoice schedule with status.

Screen 7: Reports & Analytics Dashboard



Analytics with revenue trend, collection efficiency, GST summary, and top client leaderboard.

Screen 8: Client Management



Client management with credit limit tracking, GSTIN, outstanding balance, and detailed client profile panel.

Chapter 8: Testing

8.1 Unit Testing Results

Module	Test Cases	Passed	Coverage
accounts (Auth)	14	14	97%
clients (CRUD)	16	16	95%
invoices (Engine)	22	22	94%
invoice items	12	12	96%
payments (Record)	14	14	93%
automation (Celery)	16	16	91%
PDF Generator	8	8	89%
reports	10	10	92%
Total	**112**	**112**	**93.8%**

8.2 Integration & Automation Testing

- Invoice creation API: 500 invoice creation requests processed in load test — P95 response time: 180ms.
- PDF generation: 200 PDFs generated consecutively — avg generation time: 420ms per invoice.
- Celery reminder delivery: 1,000 mock invoice reminders dispatched — 100% delivery, avg processing: 35ms/task.
- Payment webhook simulation: 500 Razorpay webhook events replayed — 100% correct status updates.
- GST calculation accuracy: 200 test invoices with various CGST/SGST/IGST scenarios — 100% accuracy vs manual calculation.
- Recurring invoice engine: 12-month simulation of 50 recurring invoices across all frequencies — 100% correct next-date calculation.

Chapter 9: Learning Outcomes

- Enterprise Business Logic: Gained deep understanding of invoice lifecycle management, GST compliance requirements, credit limit enforcement, and accounts receivable best practices from industry professionals.
- PDF Generation with ReportLab: Mastered programmatic PDF creation including canvas drawing, table rendering, image embedding, multi-page layouts, and page margins — producing pixel-perfect, print-ready invoices.
- Celery Task Automation: Developed production-grade scheduled task systems using Celery Beat for recurring billing, multi-stage reminder pipelines with conditional logic, and task retry strategies for reliable automation.
- Payment Gateway Integration: Integrated Razorpay's payment APIs, implemented webhook signature verification for secure payment callbacks, and built idempotent payment recording to prevent duplicate payment processing.
- AWS S3 Integration: Implemented secure file upload to AWS S3 using boto3, including presigned URL generation, access control configuration, and lifecycle policies for invoice PDF archival.
- Django REST Framework Advanced: Built filterable, searchable, paginated API endpoints with permission-based access control, response serializers, and comprehensive input validation.
- React.js Production Patterns: Developed complex financial data tables with sorting, filtering, real-time updates, and conditional styling based on invoice status and payment urgency.

Chapter 10: Challenges and Solutions

The development of the system involved addressing several complex technical challenges to ensure reliability, scalability, and compliance. One of the primary challenges was handling GST calculations, particularly distinguishing between intra-state (CGST + SGST) and inter-state (IGST) transactions with varying rates based on HSN codes. This was effectively resolved by implementing a dynamic TaxConfig model with a tax_type field and auto-detection logic based on GSTIN prefix comparison between the company and the client. Additionally, an HSN-based mapping table was introduced to support item-level tax customization, and the calculations were validated against real GSTR-1 sample data to ensure accuracy.

Another major issue was memory leaks during high-volume PDF generation. This was mitigated by offloading PDF creation to a dedicated Celery worker queue, ensuring isolation from the main application. Efficient memory handling techniques such as using BytesIO buffers with explicit closure after upload to **Amazon S3** were implemented. Connection pooling for the S3 client and task timeouts with retry mechanisms further improved stability and performance under load.

Task duplication in automated reminders was another challenge, especially when invoice status changed between scheduling and execution. This was resolved by adding a status validation check at the beginning of each task and implementing a distributed locking mechanism using **Redis** to prevent concurrent executions. Logging mechanisms were also introduced to track skipped tasks for auditing purposes, ensuring transparency and traceability.

Security concerns such as webhook replay attacks from **Razorpay** were handled by implementing HMAC-SHA256 signature verification and enforcing idempotency using a unique constraint on transaction references. Duplicate webhook events are safely ignored while returning appropriate HTTP responses to prevent unnecessary retries, thereby enhancing system security and robustness.

Handling Unicode characters in PDF generation was another technical hurdle, especially for Indian company names and addresses. This was resolved by registering Unicode-compatible fonts like DejaVuSans in **ReportLab**, replacing default fonts, and adding fallback mechanisms for unsupported characters. This ensured accurate and professional document rendering across different languages and character sets.

Finally, frontend performance issues were observed when handling large datasets, such as displaying over 1,000 invoices in the UI built with **React**. To address this, server-side pagination was implemented along with virtual scrolling using optimized rendering techniques. Components were memoized to reduce unnecessary re-renders, and debounced search functionality was added to minimize API calls. These optimizations significantly improved user experience by ensuring smooth and responsive interactions even with large volumes of data.

Chapter 11: Conclusion and Future Scope

11.1 Conclusion

This internship at Strivensy Technologies Pvt Ltd has been a highly productive and technically enriching experience. The Invoice Management System with Automation represents a complete, production-ready enterprise finance platform that directly addresses the pain points of Indian SMEs in invoice processing, GST compliance, and accounts receivable management.

The project successfully demonstrates that a well-designed Python full-stack application — combining Django's mature ORM and REST framework, Celery's reliable task scheduling, ReportLab's PDF generation capabilities, and React.js's interactive UI — can fully automate the invoice-to-cash cycle, reducing manual effort by an estimated 75% while improving payment collection rates through intelligent reminder automation.

11.2 Future Scope

- E-Invoicing (IRN/IRP Integration): Integrate with India's Government Invoice Registration Portal (IRP) to auto-generate Invoice Reference Numbers (IRN) and QR codes, achieving full e-invoicing compliance for taxpayers with turnover above ₹5 crore.
- AI-Powered Collection Optimization: Apply machine learning to predict which clients are likely to pay late, and dynamically adjust reminder intensity and channel (email vs SMS vs WhatsApp) based on predicted payment probability.
- Bank Statement Auto-Reconciliation: Use OCR (Tesseract/AWS Textract) to parse bank statement PDFs and automatically match bank credits to outstanding invoices, reducing manual reconciliation effort to near zero.

- Multi-Company Support: Extend the platform to support multiple legal entities under one account, enabling holding companies and CA firms to manage invoicing for multiple clients from a single interface.
- GSTR-1 Auto-Filing: Integrate with the GSTN API to directly file GSTR-1 monthly returns from invoice data, eliminating the manual GST portal data entry process.
- WhatsApp Business API Integration: Send invoice PDFs, payment links, and reminders directly via WhatsApp Business API, leveraging India's 500M+ WhatsApp users for maximum payment touchpoint coverage.

Chapter 12: References

[1] Django Software Foundation. (2024). Django Documentation v4.2. https://docs.djangoproject.com/

[2] Django REST Framework. (2024). DRF Documentation v3.14. https://www.django-rest-framework.org/

[3] Celery Project. (2024). Celery 5.3 — Distributed Task Queue. https://docs.celeryq.dev/

[4] ReportLab Inc. (2024). ReportLab PDF Generation Library. https://www.reportlab.com/docs/

[5] Razorpay Developers. (2024). Razorpay API Documentation. https://razorpay.com/docs/

[6] Stripe Inc. (2024). Stripe API Reference. https://stripe.com/docs/api

[7] Redis Labs. (2024). Redis 7 Documentation. https://redis.io/docs/

[8] Amazon Web Services. (2024). AWS S3 Developer Guide. https://docs.aws.amazon.com/s3/

[9] Python boto3. (2024). AWS SDK for Python Documentation. https://boto3.amazonaws.com/v1/documentation/

[10] React.js Team. (2024). React 18 Documentation. https://react.dev/

[11] Redux Toolkit. (2024). Redux Toolkit Official Documentation. https://redux-toolkit.js.org/

[12] PostgreSQL Global Dev Group. (2024). PostgreSQL 15 Documentation. https://www.postgresql.org/docs/

[13] Docker Inc. (2024). Docker Compose Documentation. https://docs.docker.com/compose/

[14] Government of India — GSTN. (2024). GST API Reference. https://developer.gst.gov.in/

[15] Aberdeen Group. (2019). Accounts Receivable Automation: The Path to Faster Cash. Aberdeen Research.

[16] FICCI. (2023). MSME Finance and Payment Survey — India. Federation of Indian Chambers of Commerce.

[17] Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach, 8th Ed. McGraw-Hill.

[18] Nginx Inc. (2024). Nginx Documentation. https://nginx.org/en/docs/

[19] GitHub. (2024). GitHub Actions CI/CD Documentation. https://docs.github.com/en/actions

[20] openpyxl. (2024). openpyxl Documentation. https://openpyxl.readthedos